Program Development Process

Step 1: Requirements Specification

Read the problem to be solved. State the problem clearly and understand it.

Describe the problem to be solved in your own words here:

Develop a C++ application to automate the grading of multiple-choice exams for a class of up to 30 students. The program must compare student answers against a master key, calculate individual percentages and pass/fail statuses, and generate a comprehensive 'GradeReport.txt' file that includes individual results and class-wide statistics (average, highest, and lowest scores).

Step 2: Analyze

Identify inputs, outputs, and constants. Determine what the output is first, then what inputs you need. This will lead to your list of variables and constants.

Inputs	Outputs	Constants
CorrectAnswers.txt (Answer Key)	Formatted individual results to the screen	NUM_QUESTIONS = 20
StudentList.txt (Number of students and filenames)	a final summary table to the screen	MAX_STUDENTS = 30
Individual student .txt files (Student answers)	a permanent GradeReport.txt file containing all data	

Variables List:

- char correctKey[20] (Array for the key)
- string studentFiles[30] (Array for filenames)
- int numMissed (Counter for wrong answers)
- double classAvg, int highestScore, int lowestScore (Aggregates for summary)

Step 3: Program Design

Describe the algorithm (pseudocode or flowchart) for obtaining the output from the input. Show any manual calculations here.

1. **Initialize:** Define constants and function prototypes.
2. **Load Key:** Call readFile to load CorrectAnswers.txt into an array.
3. **Load Roster:** Open StudentList.txt, read the student count, and store filenames.
4. **Loop through Students:**
    a. Attempt to open student file; if fails, skip to next.
    b. Read answers into studentAnswers array.
    c. Compare student array to key array.
    d. Record indices of wrong answers.
    e. Calculate percentage and pass/fail status.
    f. Print results to screen and append to GradeReport.txt.
    g. Update running totals for class average, high, and low scores.
5. **Finalize:** Calculate class average and write the summary footer to the screen and file.

Step 4: Implementation in C++

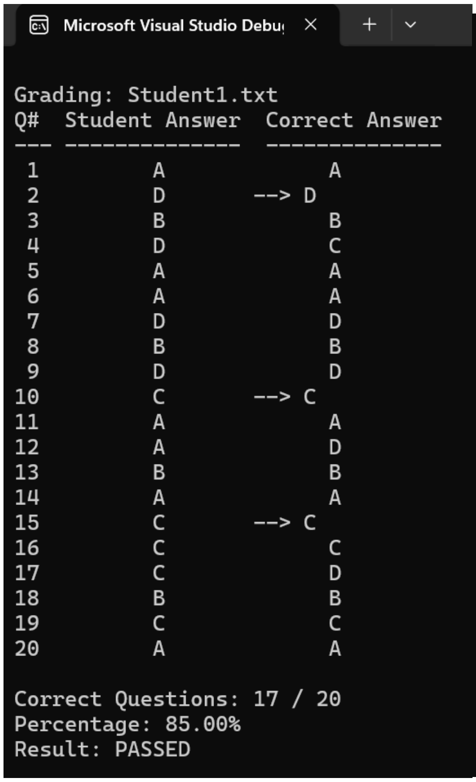
Write your declarations first, then the C++ statements. Enter and debug your program. Attach your .cpp file or paste source code here.

```
 /*******************************************************************
                         CIS-121 Spring 2026
Type of Assignment: Assn 3
Problem Number: 1
Programmer: Liliana Pantoja
Section: 400/M40
Date Due: 4/17/2026

Purpose: Develop a C++ application to automate the grading of
multiple-choice exams for a class of up to 30 students. The program
must compare student answers against a master key, calculate
individual percentages and pass/fail statuses, and generate a
comprehensive 'GradeReport.txt' file that includes individual results
and class-wide statistics (average, highest, and lowest scores).
*******************************************************************/

#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

const int NUM_QUESTIONS = 20;
const int MAX_STUDENTS = 30;

void readFile(string fileName, char answers[], int size);
void compareAnswers(char correct[], char student[], int size, int missed[], int& numMissed);
void displayResults(char correct[], char student[], int missed[], int size, int numMissed);
double calcPercentage(int numMissed, int size);
void writeReport(string studentFile, char correct[], char student[], int missed[], int
numMissed, int size, bool isFirst);

int main()
{
    char correctKey[NUM_QUESTIONS];
    string studentFiles[MAX_STUDENTS];
```

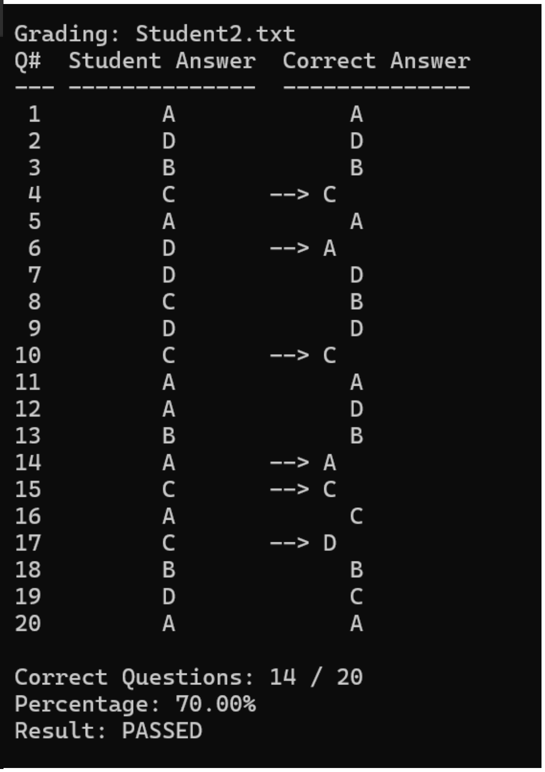
```cpp
    int numStudents;
    double totalPercentage = 0;
    int highestScore = -1, lowestScore = NUM_QUESTIONS + 1, validStudentCount = 0;
    string bestStudent, worstStudent;


    readFile("CorrectAnswers.txt", correctKey, NUM_QUESTIONS);

    ifstream listFile("StudentList.txt");
    if (!listFile) {
        cout << "Error: Could not open StudentList.txt file" << endl;
        exit(1);
    }

    listFile >> numStudents;

    if (numStudents > MAX_STUDENTS) {
        cout << "Error: Student count exceeds maximum limit of 30." << endl;
        exit(1);
    }

    for (int i = 0; i < numStudents; i++) {
        listFile >> studentFiles[i];
    }
    listFile.close();

    for (int i = 0; i < numStudents; i++) {
        char studentAnswers[NUM_QUESTIONS];
        int missedQuestions[NUM_QUESTIONS];
        int numMissed = 0;

        ifstream testFile(studentFiles[i]);
        if (!testFile) {
            cout << "\nWarning: Could not open " << studentFiles[i] << ". Skipping student." <<
endl;
            continue;
        }
        testFile.close();

        //Grade the Student
        readFile(studentFiles[i], studentAnswers, NUM_QUESTIONS);
        compareAnswers(correctKey, studentAnswers, NUM_QUESTIONS, missedQuestions, numMissed);

        //Display to screen
        cout << "\nGrading: " << studentFiles[i] << endl;
        displayResults(correctKey, studentAnswers, missedQuestions, NUM_QUESTIONS, numMissed);

        //Write to report
        writeReport(studentFiles[i], correctKey, studentAnswers, missedQuestions, numMissed,
NUM_QUESTIONS, (validStudentCount == 0));

        //Track stats
        int score = NUM_QUESTIONS - numMissed;
        double pct = calcPercentage(numMissed, NUM_QUESTIONS);
        totalPercentage += pct;
        validStudentCount++;

        if (score > highestScore) {
            highestScore = score;
            bestStudent = studentFiles[i];
        }
        if (score < lowestScore) {
            lowestScore = score;
            worstStudent = studentFiles[i];
        }
    }

    //Final Summary
    if (validStudentCount > 0) {
        double classAvg = totalPercentage / validStudentCount;

        //Display Summary on Screen
        cout << "\n=================================\n";
        cout << "CLASS GRADE REPORT\n";
        cout << "===================================\n";
        cout << fixed << setprecision(2);
        cout << "Class Average: " << classAvg << "%\n";
        cout << "Highest Score: " << highestScore << " / 20 (" << bestStudent << ")\n";
        cout << "Lowest Score: " << lowestScore << " / 20 (" << lowestScore << ")\n";
        cout << "==========================================\n";
        cout << "Grade report written to GradeReport.txt" << endl;

        //Append Summary to File
        ofstream report("GradeReport.txt", ios::app);
        report << "\n======================================\n";
        report << "CLASS GRADE REPORT\n";
        report << "=========================================\n";
        report << "Class Average: " << classAvg << "%\n";
        report << "Highest Score: " << highestScore << " / 20 (" << bestStudent << ")\n";
        report << "Lowest Score: " << lowestScore << " / 20 (" << lowestScore << ")\n";
        report << "==========================================\n";
        report.close();
    }

    return 0;
}

void readFile(string fileName, char answers[], int size) {
    ifstream file(fileName);
    if (!file) {
        cout << "Error: Could not open " << fileName << endl;
        exit(1);
    }
    for (int i = 0; i < size; i++) {
        file >> answers[i];
    }
    file.close();
}

void compareAnswers(char correct[], char student[], int size, int missed[], int& numMissed) {
    numMissed = 0;
    for (int i = 0; i < size; i++) {
        if (student[i] != correct[i]) {
            missed[numMissed] = i - 1;
            numMissed++;
        }
    }
}

double calcPercentage(int numMissed, int size) {
    return ((double)(size - numMissed) / size) * 100.0;
}

void displayResults(char correct[], char student[], int missed[], int size, int numMissed) {
    int missedIdx;
    double pct;

    cout << "Q#  Student Answer  Correct Answer" << endl;
    cout << "--- --------------  --------------" << endl;
    missedIdx = 0;
    for (int i = 0; i < size; i++) {
        cout << setw(2) << (i + 1) << "         " << student[i];

        if (missedIdx < numMissed && missed[missedIdx] == (i + 1)) {
            cout << "       --> " << correct[i] << endl;
            missedIdx++;
        }
        else {
            cout << "             " << correct[i] << endl;
        }
    }

    pct = calcPercentage(numMissed, size);
    cout << "\nCorrect Questions: " << (size - numMissed) << " / " << size << endl;
    cout << fixed << setprecision(2) << "Percentage: " << pct << "%" << endl;
    cout << "Result: " << (pct >= 70.0 ? "PASSED" : "FAILED") << endl;

}

void writeReport(string studentFile, char correct[], char student[], int missed[], int
numMissed, int size, bool isFirst) {
    ofstream report;

    if (isFirst) {
        report.open("GradeReport.txt", ios::trunc);
    }
    else {
        report.open("GradeReport.txt", ios::app);
    }

    report << "\nGrading: " << studentFile << endl;
    report << "Q#  Student Answer  Correct Answer" << endl;
    report << "--- --------------  --------------" << endl;

    int missedIdx = 0;
    for (int i = 0; i < size; i++) {
        report << setw(2) << (i + 1) << "           " << student[i];
        if (missedIdx < numMissed && missed[missedIdx] == (i + 1)) {
            report << "         --> " << correct[i] << endl;
            missedIdx++;
        }
        else {
            report << "           " << correct[i] << endl;
        }
    }

    double pct = calcPercentage(numMissed, size);
    report << "\nCorrect Questions: " << (size - numMissed) << " / " << size << endl;
    report << fixed << setprecision(2) << "Percentage: " << pct << "%" << endl;
    report << "Result: " << (pct >= 70.0 ? "PASSED" : "FAILED") << endl;
    report << "----------------------------------------" << endl;

    report.close();
}
```

Step 5: Testing

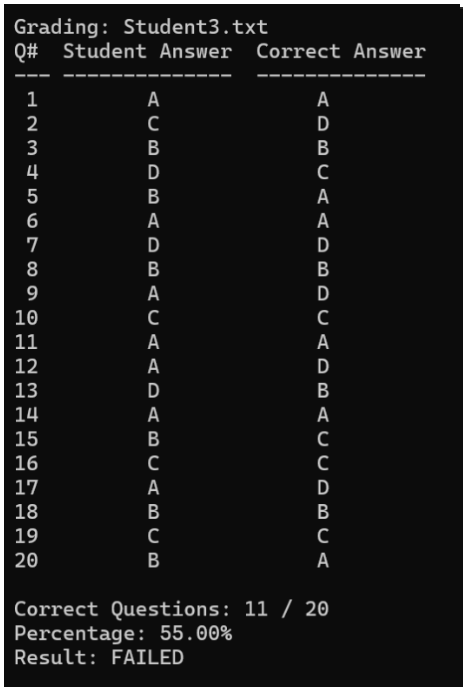
Test with multiple data sets including edge cases (e.g., all correct, all wrong, exactly 70%). Summarize and analyze your results. Paste output screenshots here.



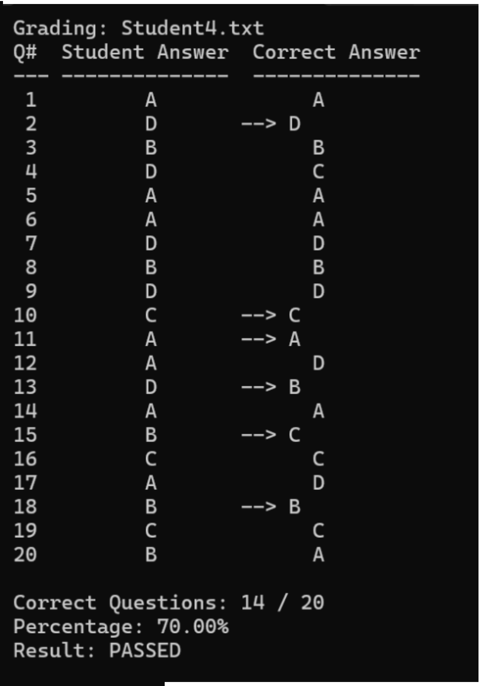
```
Grading: Student1.txt
Q#  Student Answer  Correct Answer
--- --------------  --------------
 1         A             A
 2         D       --> D
 3         B             B
 4         D             C
 5         A             A
 6         A             A
 7         D             D
 8         B             B
 9         D             D
10         C       --> C
11         A             A
12         A             D
13         B             B
14         A             A
15         C       --> C
16         C             C
17         C             D
18         B             B
19         C             C
20         A             A

Correct Questions: 17 / 20
Percentage: 85.00%
Result: PASSED
```



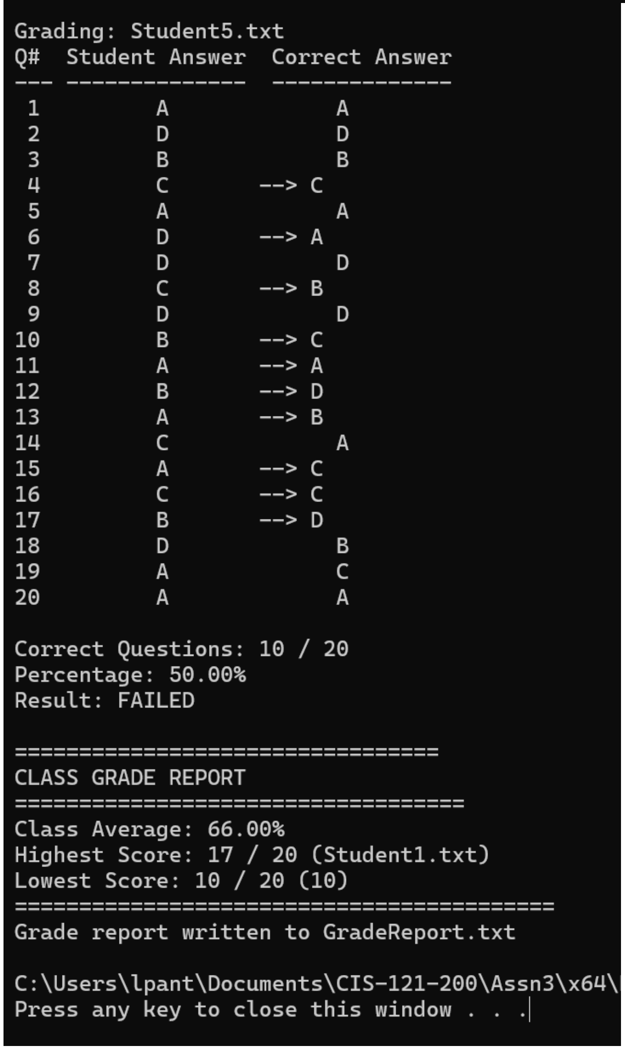
```
Grading: Student2.txt
Q#  Student Answer  Correct Answer
--- --------------  --------------
 1         A             A
 2         D             D
 3         B             B
 4         C       --> C
 5         A             A
 6         D       --> A
 7         D             D
 8         C             B
 9         D             D
10         C       --> C
11         A             A
12         A             D
13         B             B
14         A       --> A
15         C       --> C
16         A             C
17         C       --> D
18         B             B
19         D             C
20         A             A

Correct Questions: 14 / 20
Percentage: 70.00%
Result: PASSED
```



```
Grading: Student3.txt
Q#  Student Answer  Correct Answer
--- --------------  --------------
 1         A             A
 2         C             D
 3         B             B
 4         D             C
 5         B             A
 6         A             A
 7         D             D
 8         B             B
 9         A             D
10         C             C
11         A             A
12         A             D
13         D             B
14         A             A
15         B             C
16         C             C
17         A             D
18         B             B
19         C             C
20         B             A

Correct Questions: 11 / 20
Percentage: 55.00%
Result: FAILED
```



```
Grading: Student4.txt
Q#  Student Answer  Correct Answer
--- --------------  --------------
 1         A             A
 2         D       --> D
 3         B             B
 4         D             C
 5         A             A
 6         A             A
 7         D             D
 8         B             B
 9         D             D
10         C       --> C
11         A       --> A
12         A             D
13         D       --> B
14         A             A
15         B       --> C
16         C             C
17         A             D
18         B       --> B
19         C             C
20         B             A

Correct Questions: 14 / 20
Percentage: 70.00%
Result: PASSED
```



```
Grading: Student5.txt
Q#  Student Answer  Correct Answer
--- --------------  --------------
 1         A             A
 2         D             D
 3         B             B
 4         C       --> C
 5         A             A
 6         D       --> A
 7         D             D
 8         C       --> B
 9         D             D
10         B       --> C
11         A       --> A
12         B       --> D
13         A       --> B
14         C             A
15         A       --> C
16         C       --> C
17         B       --> D
18         D             B
19         A             C
20         A             A

Correct Questions: 10 / 20
Percentage: 50.00%
Result: FAILED

=================================
CLASS GRADE REPORT
===================================
Class Average: 66.00%
Highest Score: 17 / 20 (Student1.txt)
Lowest Score: 10 / 20 (10)
==========================================
Grade report written to GradeReport.txt

C:\Users\lpant\Documents\CIS-121-200\Assn3\x64\
Press any key to close this window . . .
```